POORNIMA

DSA (Java)

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

* Code errors → google (stack overflow website)
* work hard & solve your own problems

* flowcharts :-

Diagram to represent solutions of problems.

① small parts

↓

② logically arrange

* Components -

Start / Exit — Start → oval

Input / Output — Read N — Print "Hi" → parallelogram

Process — name = "shradha" → Rectangle

→ assignment operator

Decision — is marks > 33 ? → diamond

Arrows

↓

★ Sum of two numbers

Input

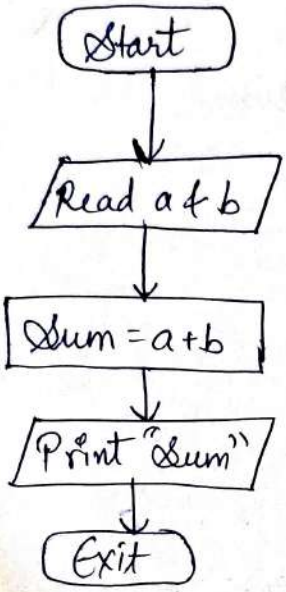
no. 1 = a

no. 2 = b

Output

Sum of a & b

Flowchart



Pseudocode

1.) Start
2.) Input numbers, a & b
3.) Calculate Sum = a + b
4.) Print Sum
5.) Exit

* Pseudocode is irrespective of programming language

★ Calculate Simple Interest

Input

Principal = P

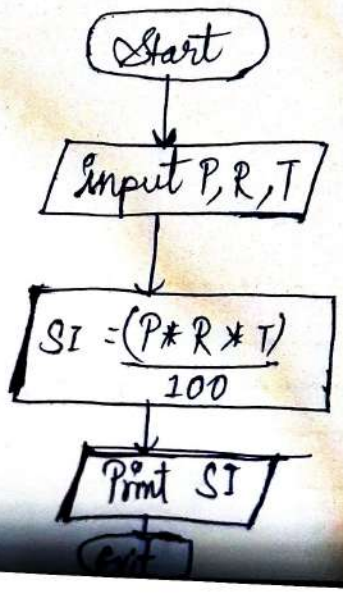
Rate = R

Time = T

Output

$S.I. = \frac{P \times R \times T}{100}$

flowchart



Pseudocode

1.) Start
2.) input values, P, R and T
3.) Calculate SI = (P * R * T) / 100
4.) Print Simple Interest
5.) Exit

Date	Unit No.	Lecture No.	Faculty			

★ find max of 3 numbers -

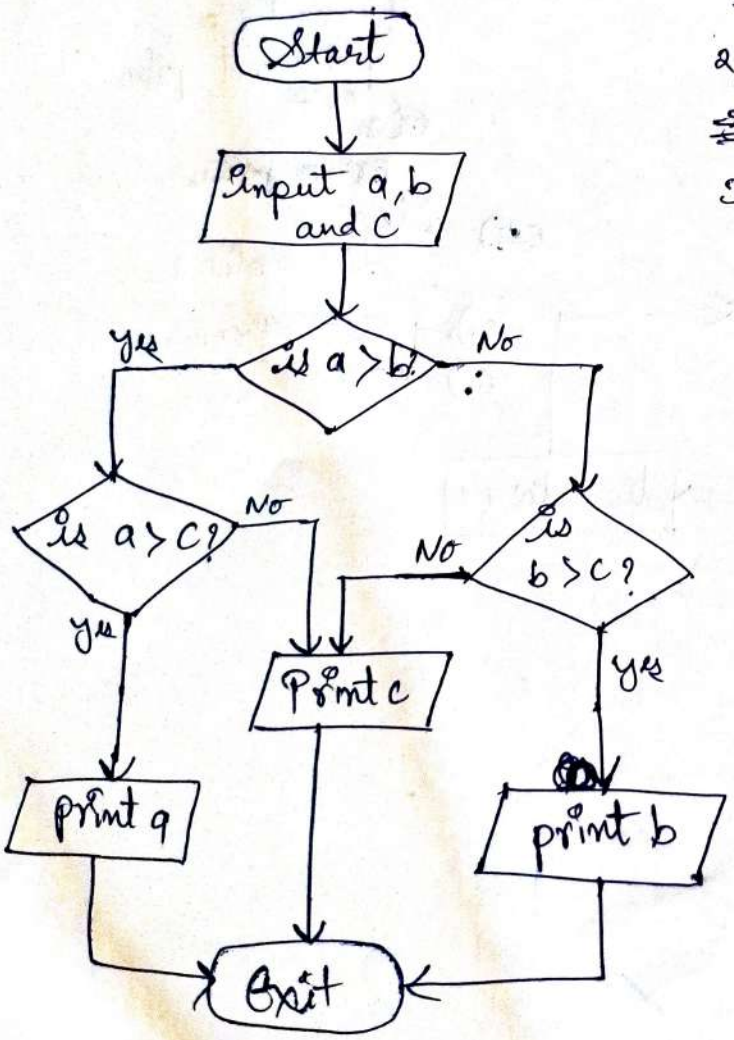
Input

three nos.
a, b & c

Output

max of 3

Flowchart



Pseudocode

1.) Start
2.) input three nos. a, b & c
~~3.) Compare~~
3.) if a > b do
    if a > c do
        print a
    else
        print c
else
    if b > c do
        print b
    else
        print c
4.) Exit

★ find if no. is prime -

Input

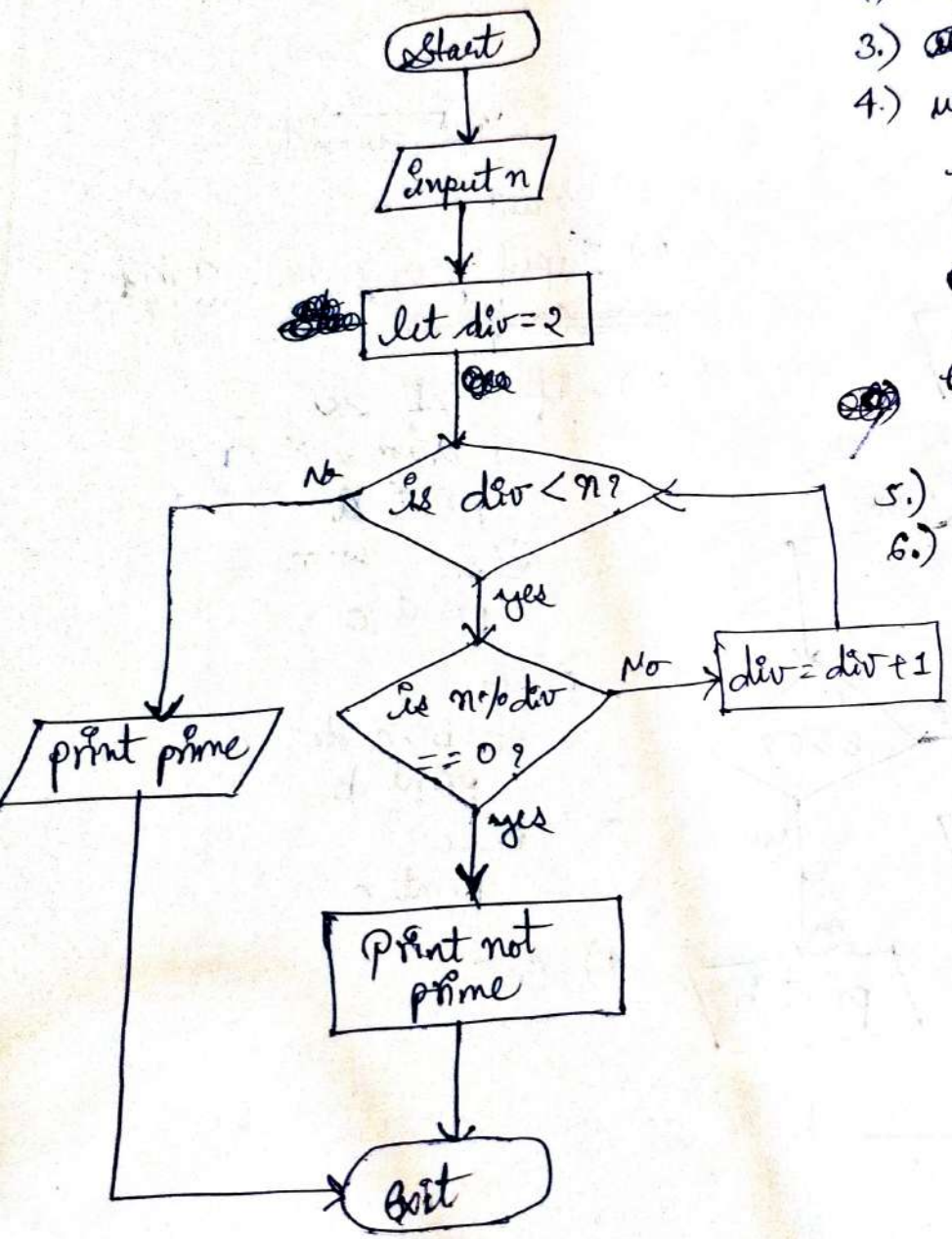
a no. = n

Output

prime or not prime

* any non prime no. must have divisor b/w 2 to n-1

flowchart



pseudocode

1.) Start
2.) Input n
3.) let div = 2
4.) while div < n do
    if n % div == 0
        print not prime
        exit
    else
        div = div + 1
5.) Print Prime
6.) exit

POORNIMA

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

★ Sum of first n natural nos. –

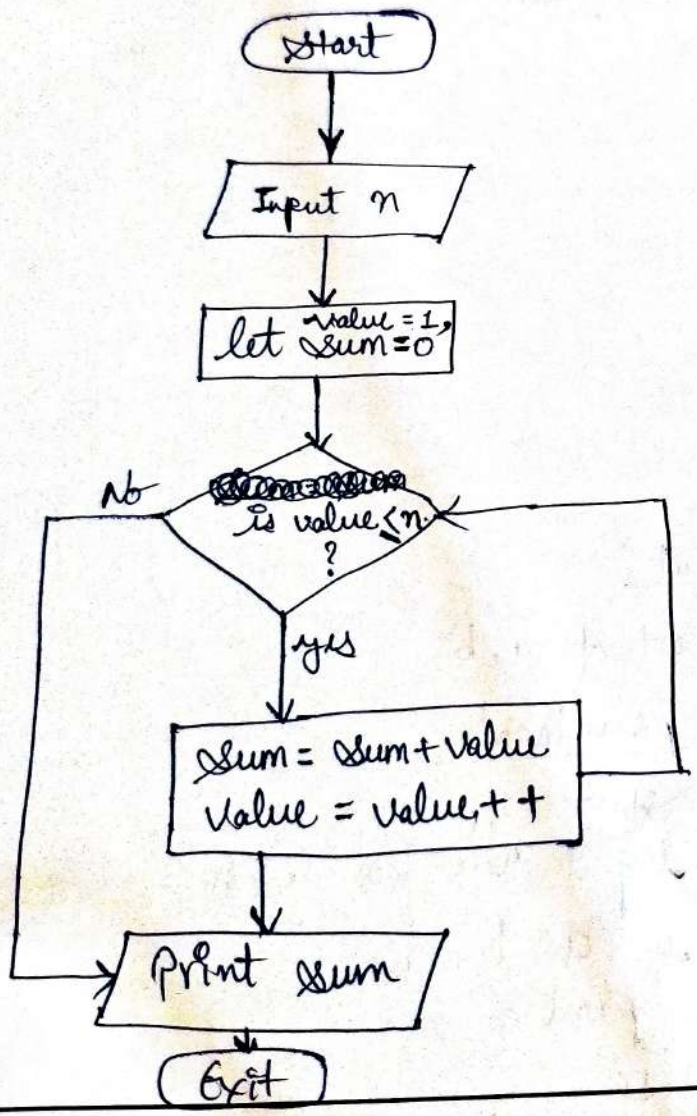
Input

n : no. of terms

Output

Sum of n natural nos.

Flowchart



Pseudocode

1.) Start
2.) Input n
3.) let sum = 0 and value = 1
4.) while value ≤ n do
    sum = sum + value
    value = value + 1
5.) print sum
6.) Exit

Main Ideas, Questions & Summary:

Library / Website Ref.:-

Ques 1 Area of a circle

Ans

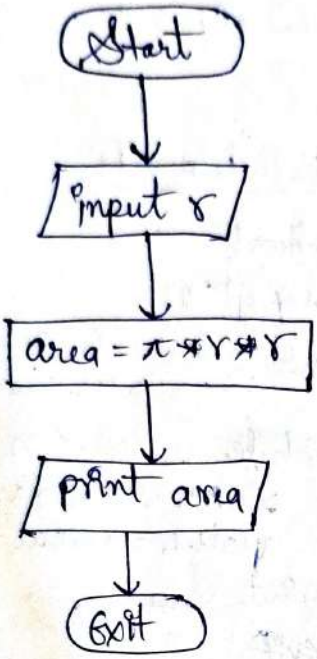
Input

radius = r

Output

area = πr^2

flowchart



pseudocode

1.) Start
2.) Input r
3.) Calculate area = π*r*r
4.) print area
5.) exit

Que. 2 greatest of 2 nos.

Ans

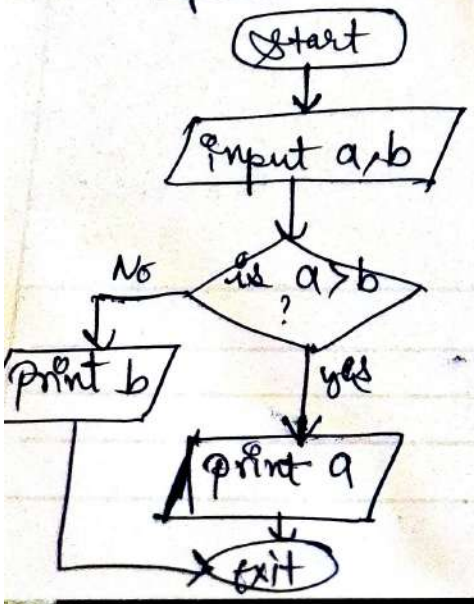
Input

2 nos. a & b

Output

greatest of a, b

flowchart



pseudocode

1.) Start
2.) Input two nos. a & b
3.) if a > b do
print a
else
print b
4.) Exit

POORNIMA

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

Que. 3 print Even nos. b/w 9 & 100.

Ans

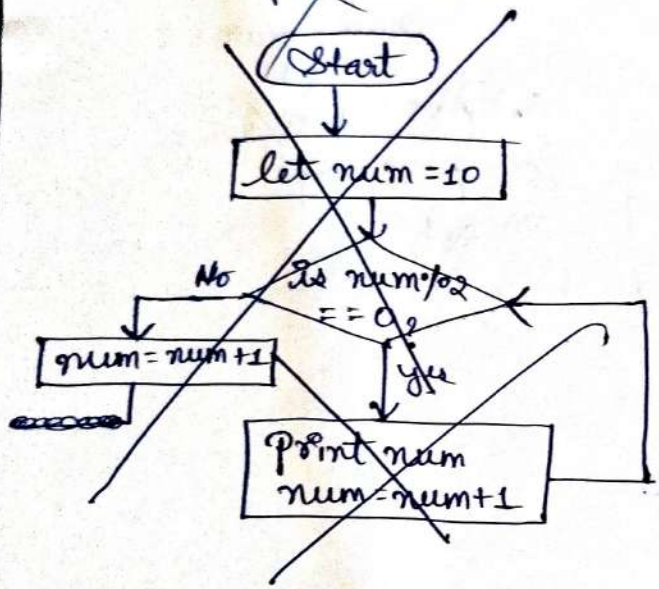
Input

Range 9 to 100

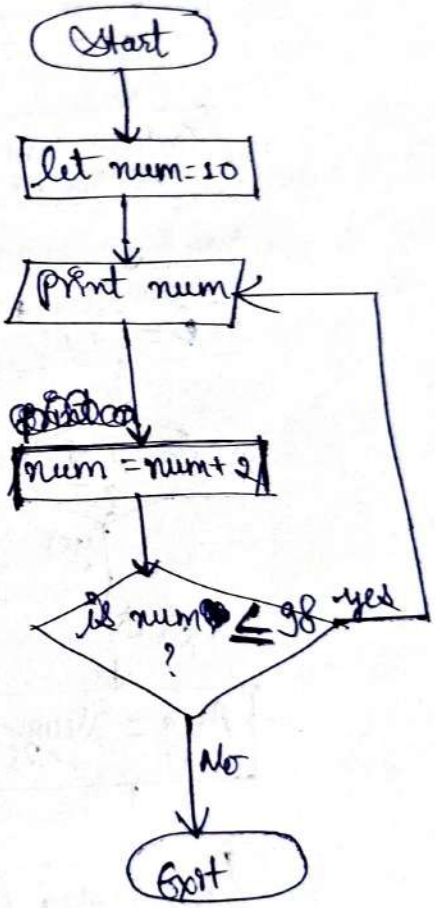
Output

even nos. b/w 9 to 100

~~flowchart~~



flowchart



pseudocode

1.) Start
2.) let num = 10
3.) print num
4.) num = num + 2
5.) if num ≤ 98
print num
else
exit
6.) exit

Main Ideas, Questions & Summary:

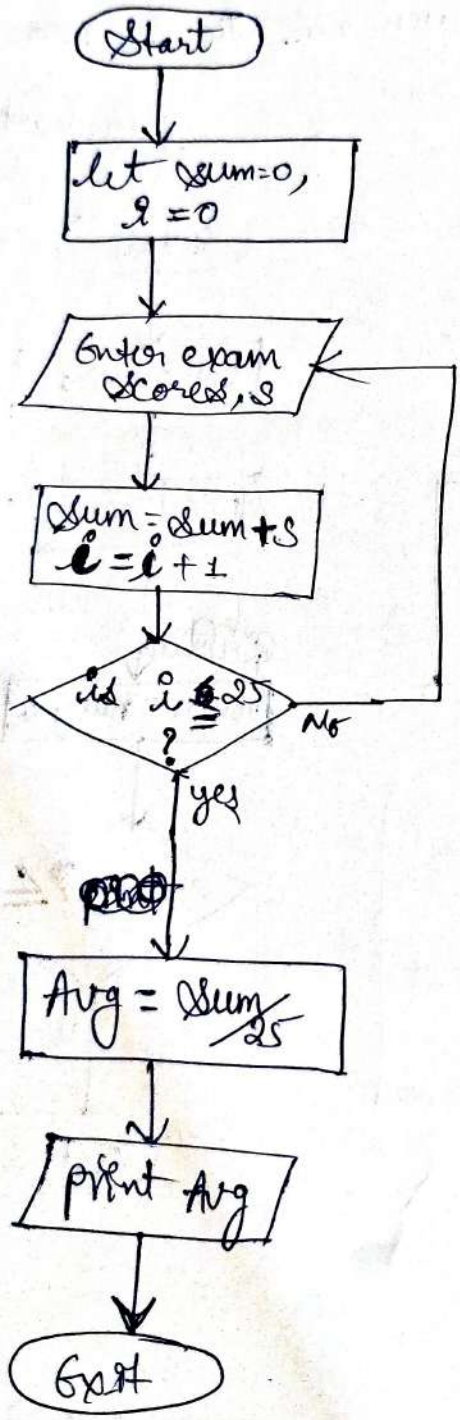
Library / Website Ref.:-

Que.4. calculating the average from 25 exam scores.

Ans

Input	Output
25 exam scores	average

flowchart



pseudocode

1.) Start
2.) let sum = 0 and i = 0
3.) Enter exam scores s
4.) while i ≤ 25
    sum = sum + s
    i = i + 1
5.) Avg = sum/25
6.) print Avg
7.) Exit

POORNIMA

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

* In java public class name & file name must be same.

* <u>Boiler plate code</u> :-

```
public class Main{
    public static void main (String args[]) {
        :
    }
}
```

* <u>Output In Java</u> :-

System.out.<u>print</u> (<u>"Hello world"</u>); → statement terminator

print → fn, "Hello world" → output

ln → used for next line

\n → ——"——

1.) print :-

System.out.print ("Hello world");

→ string

2.) println :- System.out.println ("Hello World!");

→ next line

3.) "\n"

* Variables in Java :—

- **literals** :— 1, 2, 3 etc
  'a', 'b', 'c' etc
  '@', '*' etc

- variables :— a, b etc.

memory

a	b	name
5	10	"anushka"

a, b, c
file name
fn name
etc
} → Identifiers

* Data Types in Java :—

* Java is a typed language ⇒ you have to declare type of data before creating variable

Data Types

Primitive (8)

byte - 1 byte
short - 2 —"—
char - 2 —"—
boolean - 1 —"—
int - 4 —"— (-2 billion to +2 billion)
long - 8 —"—
float - 4 —"—
double - 8 —"—

Non - Primitive

String
Array
class
Object
Interface

- byte range upto 256 nos. (-128 to 127)
- int > short > byte
  4 byte > 2 byte > 1 byte

POORNIMA

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

★ Comments :- Compiler ignores it ; only for humans.

// Hi Anushka → single line comments

/* Hi Anushka
How are you? */ → Multi line comments

★ Input In Java -

Scanner - It is a class which already exists in Java
Scanner sc = new Scanner (System.in);
import java.util.* ; → util package in Java
sc.next (); - for input

.next () → only before space
.nextline () → complete string (space included)
.nextInt () → Integer input
.nextFloat () → float input

★ Type Conversion (widening Conversion)/ implicit conversion -
~~Conversio~~ Conversion happens when :
① types are compatible
② destination type > source type

byte → short → int → float → long → double

ain Ideas, Questions & Summary:

★ Type casting / narrowing conversion / explicit conversion :-

Just write the type you wanna convert to

```
float a = 9.99f;
int b = (int) a;
```

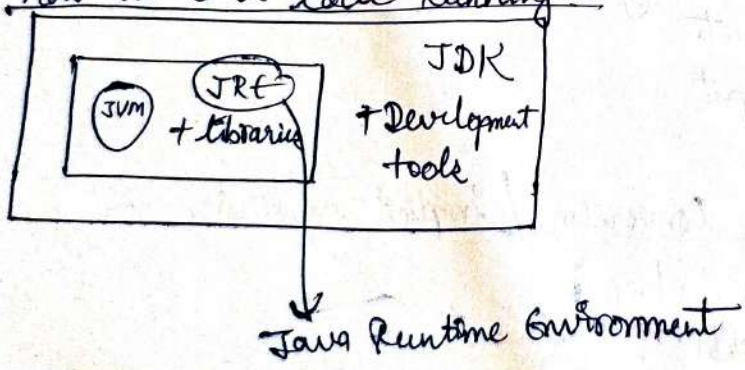
- every character has their own ASCII value i.e. numeric value.
  'a' = 97 'b' = 98 etc...

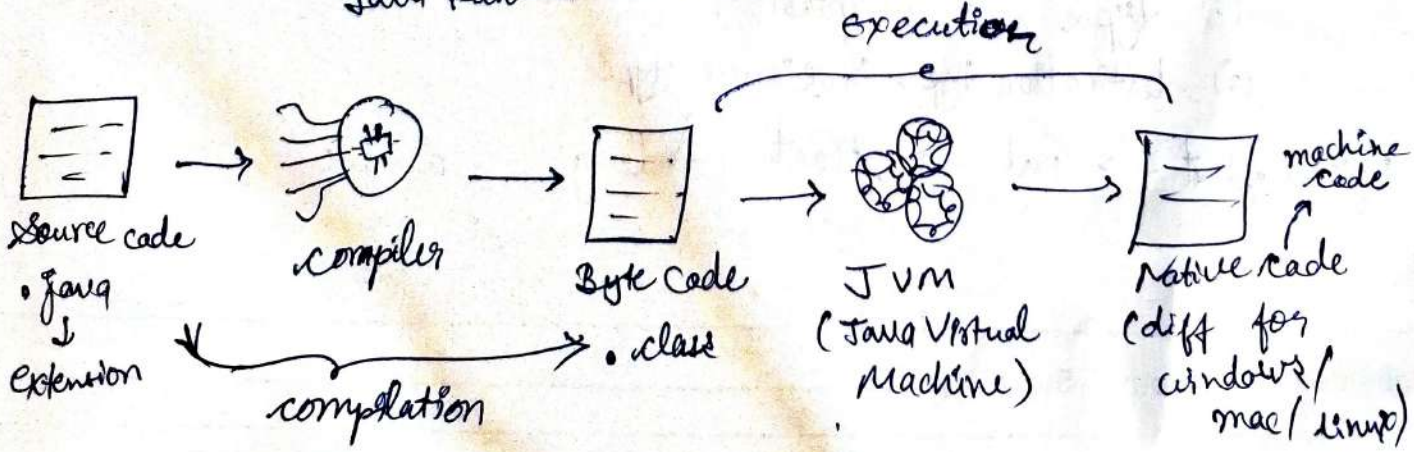
★ Type promotion in Expressions:-

1. Java Promote byte, short & char to int automatically.
   → only in expressions.

2. if largest possible datatype in an expression is long then all the variables get converted into long.
   Same with float & double.
   ~~Jeno~~ hai to double.

★ How is our code Running:-





- Java is portable language => i.e. systems independent

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

* variable (identifier) name can include letters, digits (but not at the start), underscore, $
  only underscore not possible for Java 8 + version

* **Operators in Java:-**

Sum = a + b → expression
a → operands
+ → operator

* operators are the symbols that tell compiler to perform some operation.

* **Types of operators:-**

1.) Arithmetic operators (Ternary / Unary / Binary)
2.) Relational operators
3.) Logical operators
4.) Bitwise operators (Bit Manipulation)
5.) Assignment operators

(1) Arithmetic operators -

→ Binary : they have 2 operands
Ex:- + - * / % → Remainder
Modulo → quotient

→ Unary : they require only 1 operand
Ex:- ++ --

Main Ideas, Questions & Summary:

* ++ - increment
* -- → decrement

• pre increment / decrement
++a / --a
→ 1.) value change
2.) value use

• Post increment / decrement
a++ / a--
1.) value use
2.) Value change

* Relational operators :-

① == equal to (to check if values are equal)
② != not equal to
③ > greater than
④ < less than
⑤ >= greater than equals to
⑥ <= less than equals to

→ This prints boolean true or false

* logical operators :-

① && (logical AND) → gives true when both statements are true
② || (logical OR) → gives true when at least one statement is true
③ ! (logical NOT) → invert the statements

* Assignment operators :-

① = (assign value of righ side to left)
a = a+10 ⇒ a += 10 → faster execution (nanoseconds change hoti but still hai)

② +=
③ -=
④ *=
⑤ /=
⑥ %=

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

★ IDE → Integrated Development Environment
→ all essentials tools like code editor, compiler, debugger etc. are integrated together.

Ex:- Intellij IDEA
VS code + extensions = IDE

★ <u>Operator Precedence</u> :-

Operator precedence determines the order in which the operators in an expression are evaluated.

Operator	Precedence
Postfix	++ --
prefix & unary (→ right to left)	++, --, + (unary +), − (unary −), ~, !
multiplicative	*, /, %
additive	+ −
shift	<< (x<<n = x × 2^n), >> (sign), >>> (no sign) (x>>n = x ÷ 2^n)
relational	> < <= >= instanceof (check whether an object belongs to a specific class or subclass returns boolean)
equality	== !=
bitwise AND	&
bitwise exclusive OR	^
bitwise inclusive OR	|
logical AND	&&
logical OR	||

a=10
b=−a
b=−10

Main Ideas, Questions & Summary:

Library / Website Ref.:-

ternary → right to left ?:

assignment = += -= *= %= /= &=

↳ they work right to left ^= |= <<= >>= >>>=

a=b=c=5
i.e. c=5
b=c
a=b

* Conditional Statements :-

① if - else statements

syntax:

```
if (condition){
}
else {
}
```

→ In Java else always belongs to the nearest if that does not already have an else.

② else - if statements

```
if (condition 1) {
}
else if (condition 2){
}
else {
}
```

→ else if block can be multiple.

* Ternary operator :-

operates on 3 operands.

Used to write if - else in one i.e. compact form.

Date No.	Faculty	Subject Name	Subject Code	Main Topics:-

variable = condition ? Statement 1 : Statement 2;

result from st1 / 2st stores in variable.

③ Switch Statement :-

```
switch (variable) {
case 1:
case 2:
case 3:
default:
}
```

→ Variable inse compare hota hai

if one case is matched all the other case after that will also be implemented. to prevent this use 'break'.

→ order can be changed. They can be char as well, float, double as well.

* loops :-

* while loop -

```
while (condition) {
  ------
}
```

→ The loop runs until the condition is true.

"don't get stuck in infinite loops".

* i = iterator

Main Ideas, Questions & Summary:

Library / Website Ref.:-

★ for loop :-

for (initialisation ; condition ; updation){

}

get last digit of any no. = no % 10
remove last digit of any no. = n/10

★ do while loop :-

do {

} while (condition);

★ Break statement :- to exit from a loop.

★ continue statement :- to skip an iteration

Math.sqrt (n) = $\sqrt{n}$

★ scope of a variable is referred to the part of the program where the variable can be used.
for ex:- If 'i' is declared inside a for loop.

for (int i = 1; — ; — ;){

}

Then it can only be used inside for loop & not outside.

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

* nested loops :-
making a loop inside another loop

```
• if (a>b){
      if (a>c){   ——→ nested if-else
      }
  }
```

```
• for (int i=0; i<n; i++){
      for (int j=0; j<n; j++){   ——→ nested loops
      }
  }
```

• outer loop runs the no. of rows (lines)
• inner loop → columns (no. of times)
• what to print (ex:- *)

* inverted star:-

```
for (int i=1; i<=4; i++){
    for(int j=4; j>=i; j--){
        System.out.print("* ");
    }
    System.out.println();
}
```

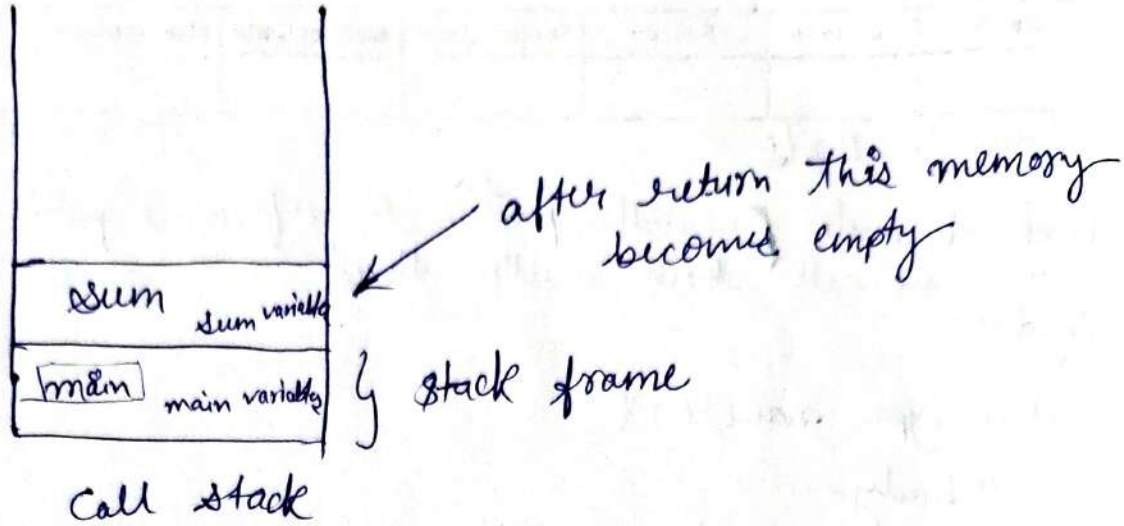
★ Half pyramid pattern:-

```
for (int i=1; i≤4; i++){
    for (int j=1; j≤i; j++){
        System.out.print(j);
    }
System.out.println();
}
```

★ character pattern:-

```
char ch = 'A';
for (int i=1; i≤4; i++){
    for (int j=1; j≤i; j++){
        System.out.print(ch);
        ch++;
    }
System.out.println();
}
```

* functions occupy memory in call stack.



* Java always calls by value
i.e. if you pass a value in a function, the original value doesn't pass, the copy of that original variable is passed to another function.

* original value => call by reference
✗ not in java

* helper functions can be created.
other functions (than main) can also call functions.

* Methods

user defined
ex:- factorial
sum
product

Inbuilt method
ex:- Math → pow
→ sqrt
→ max

nextInt();

POORNIMA

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

* function overloading :-

→ Multiple functions with the same name but different parameters in the same class.

→ we can vary type of parameters or no. of parameters or both.

→ It doesn't depends on return type.

* convert from Binary to decimal & vice versa :-

$(0)_{10} = (0)_2$

$(1)_{10} = (01)_2$

$(2)_{10} = (10)_2$

$(3)_{10} = (11)_2$

$(4)_{10} = (100)_2$

$(5)_{10} = (101)_2$

$(6)_{10} = (110)_2$

$(7)_{10} = (111)_2$

$(8)_{10} = (1000)_2$

Binary to decimal :-

$\bar{X}2^2 \; \bar{Y}2^1 \; \bar{X}2^0$ ...

$= 1 \; 1 \; 1$

decimal to binary :-

2	n = 7	1
2	3	1
2	1	1
	0	

→ last wala first

Main Ideas, Questions & Summary:

* Scope :-

where can you use the variable you defined.

★ Method Scope :-

It tells that if you have defined a variable inside a method you can use that variable anywhere inside the method but after the definition line & not before.

But you can't directly use that methods variable into another method directly. the only way to use is by passing parameters.

★ Block Scope :-

(1) Block of code (5) } variables can only be used inside that block only

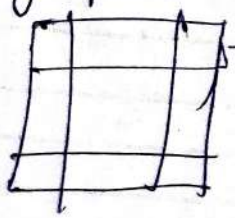
★ class Scope :-

access modifiers

* Hollow rectangle Pattern :-

identify patterns

ex:-



boundaries are filled

so, m = 1 & last

n = 1 & last

print (*)

Date	Unit No.	Lecture No.	Faculty	Subject Name	Subject Code	Main Topics:-

* Inverted & Rotated Pyramid :-

```
        *
      * *
    * * *
  * * * *
```

logic :- print spaces = $n-i$
print stars = i
} two inner loops

* Inverted Half-Pyramid with nos. :-

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

logic :- $j=1$ to $n-i+1$
print j

* Floyd's triangle :-

```
1
2 3
4 5 6
9 8 9 10
11 12 13 14 15
```

logic :- just make a tracker
num = 1
increase num++ after every time you print it.

P.T.O.

* 0-1 Triangle :-

```
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
```

logic :- if row = even & col = odd
(or)
row = odd & col = even
print "0"
else print "1".

better logic :- if i+j = even print 1
if i+j = odd print 0

* Butterfly pattern :-

* Butterfly pattern :- ex :- n=4

```
*       *
* *     * *
* * *   * * *
* * * * * * * *
* * * * * * * *
* * *   * * *
* *     * *
*       *
```

logic :-
first half :- outer loop 1 to n
inner loop :-

Stars	spacer	Stars
i	(n-i)*2	i

second half :-
outer loop n to 1 (mirror image)
inner loop same

Subject Code | Main Topics:-

* Solid Rhombus :-

```
    * * * * *
   * * * * *
  * * * * *
 * * * *
* * * *
```

$n = 5$
$i = 5$

logic :- Spaces Stars
$n - i$ n

* Hollow Rhombus :-

```
    * * * * *
   *       *
  *       *
 *       *
*  * * * *
```

logic :- $n = 5$
$i = 5$

spaces + boundary of rhombus
$n - i$

if $i = 1$ || $i = n$ ||
$j = 1$ || $j = n$
print *

* Diamond pattern :-

```
      *
     * * *
    * * * * *
   * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

$n = 4$

logic :- $n = 4$
$i = 4$

Space + Star
$n - i$ $2i - 1$

when printing patterns, you have to ignore the spaces in the end but have to print (explicitly) space in the start.